

Department of Information Engineering Technology
The Superior University, Lahore

Mobile Application Development Lab

Experiment No.11

Flutter: API integration

Prepared for

By:

Name: _____

ID: _____

Section: _____

Semester: _____

Total Marks: _____

Obtained Marks: _____

Signature: _____

Date: _____

Experiment No.11
Flutter: API integration
(Rubrics)

Name: _____

Roll No: _____

A. PSYCHOMOTOR

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Follow Procedures	1	0	0.25	0.5	0.75	1	
2	Software and Simulations	2	0	0.5	1	1.5	2	
3	Accuracy in Output Results	3	0	0.75	1.5	2.25	3	
Sub Total		6	Sub Total marks Obtained in Psychomotor(P)					

B. AFFECTIVE

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Respond to Questions	1	0	0.25	0.5	0.75	1	
2	Lab Report	1	0	0.25	0.5	0.75	1	
3	Assigned Task	2	0	0.5	1	1.5	2	
Sub Total		4	Sub Total marks Obtained in Affective (A)					

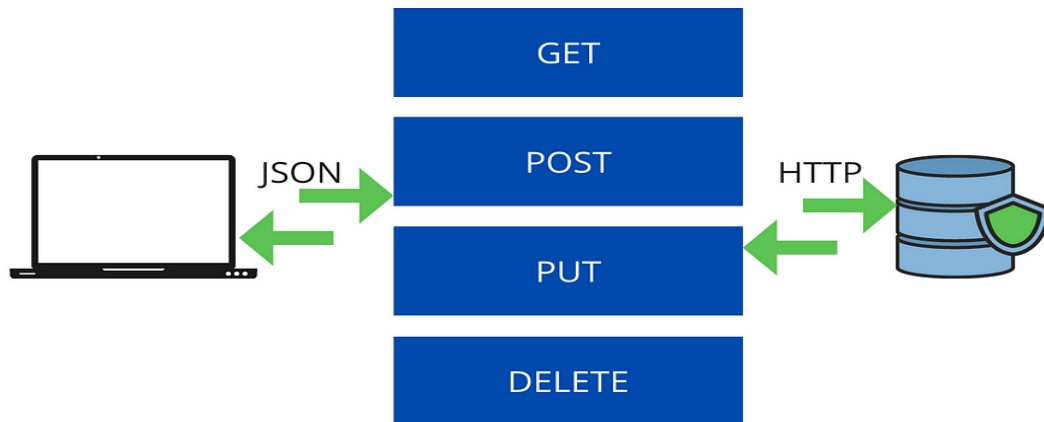
Instructor Name: _____

Total Marks (P+A): _____

Instructor Signature: _____

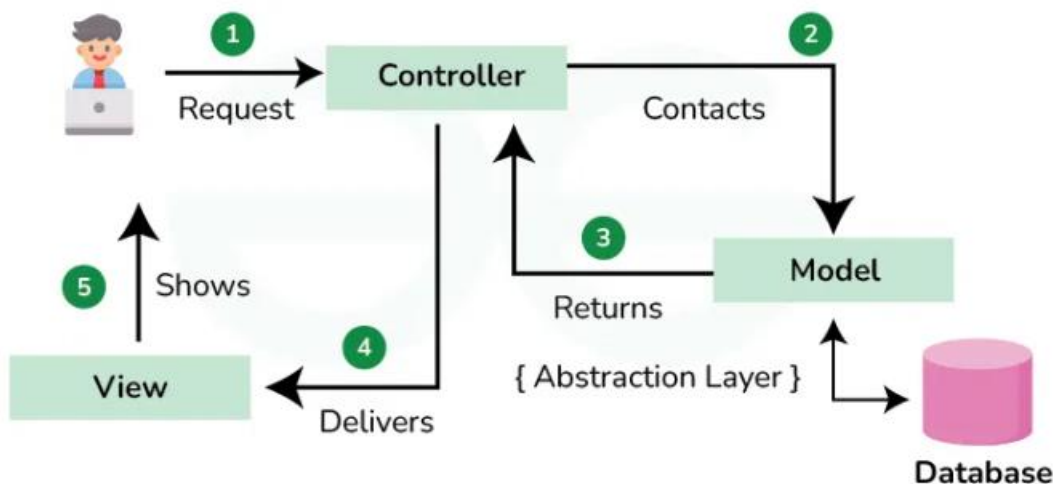
Obtained Marks (P+A): _____

11.1 How do APIs Work?



A user application can contact any database through HTTP requests. So, we can create GET, POST, PUT, or DELETE HTTP requests to a database. In return, the database sends us data or results or responses in form of JSON or HTML or XML. The JSON format is widely used. We then parse the JSON into a proper model class and use it in our app.

11.2 Integrate APIs into Flutter App



To integrate an API, we have a few steps that we can follow for our ease:

Step 1: Get API URL and endpoints.

Step 2: Add relevant packages into the app (http, dio, chopper, etc.).

Step 3: Create a constant file that stores URLs and endpoints.

Step 4: Create a model class to parse the JSON.

Step 5: Create a file that handles the API call and write specific methods to fetch data and parse it.

Step 6: Use the data in your app.

We will use [JSONPlaceholder](#) API which provides fake APIs for us to practice.

Step 1: Get API URL and endpoints.

To get the API URL (referred to as base URL) and endpoints, go to the [JSONPlaceholder](#) website and you will find different APIs that they provide. We would today use the users API that they have. Hence,

Base URL: `https://jsonplaceholder.typicode.com`

API Endpoint: `/users`

When you have a database or API provider, the base URL for all APIs remain the same however the endpoint changes depending on the API. In many API Providers, you will have to obtain a private key (API Key or Access Key) either by simply creating an account or it can be a paid one. That API Key will be just appended to the base URL. So the new base URL will be:

New Base URL = `BaseUrl/apiKey`

Example: `https://jsonplaceholder.typicode.com/abcdefghijklmnopqrstuvwxyz`

Step 2: Add relevant packages into the app (http, dio, chopper, etc.)

There are many packages available in [pub.dev](#) that we can use to integrate APIs in Flutter. The most widely used packages are:

- [http](#)
- [dio](#)

- chopper

And many more... However, http is the most basic and easy to use. The other packages are mostly the wrapper of the http package and provide additional functionalities.

Now, once you have created a new flutter project, go to pubspec.yaml file and add the http package into it. Your file should look something like the following:

```
1  name: rest_api_example
2  description: A new Flutter project.
3  publish_to: 'none' # Remove this line if you wish to publish to pub.dev
4  version: 1.0.0+1
5
6  environment:
7    sdk: ">=2.15.1 <3.0.0"
8
9  dependencies:
10   cupertino_icons: ^1.0.2
11   flutter:
12     sdk: flutter
13   http: ^0.13.4
14
15  dev_dependencies:
16   flutter_lints: ^1.0.0
17   flutter_test:
18     sdk: flutter
19
20  flutter:
21   uses-material-design: true
```

Step 3: Create a constant file that stores URLs and endpoints.

Now, it's time to create a simple file named `constants.dart` that will hold all your URLs and endpoints. In our case, we only have one endpoint, but it's a good practice to have a separate file. Your file will look something like the following:

```
1  class ApiConstants {
2    static String baseUrl = 'https://jsonplaceholder.typicode.com';
3    static String usersEndpoint = '/users';
4  }
```

Here, we have created a class called ApiConstants and created 2 static variables so that we can access them without creating an instance of the class like `ApiConstants.baseUrl` .

Step 4: Create a model class to parse the JSON.

One way to access the data is directly using the key. However, it's a good and easy approach to create a model class, parse the JSON and get an object out of that JSON response.

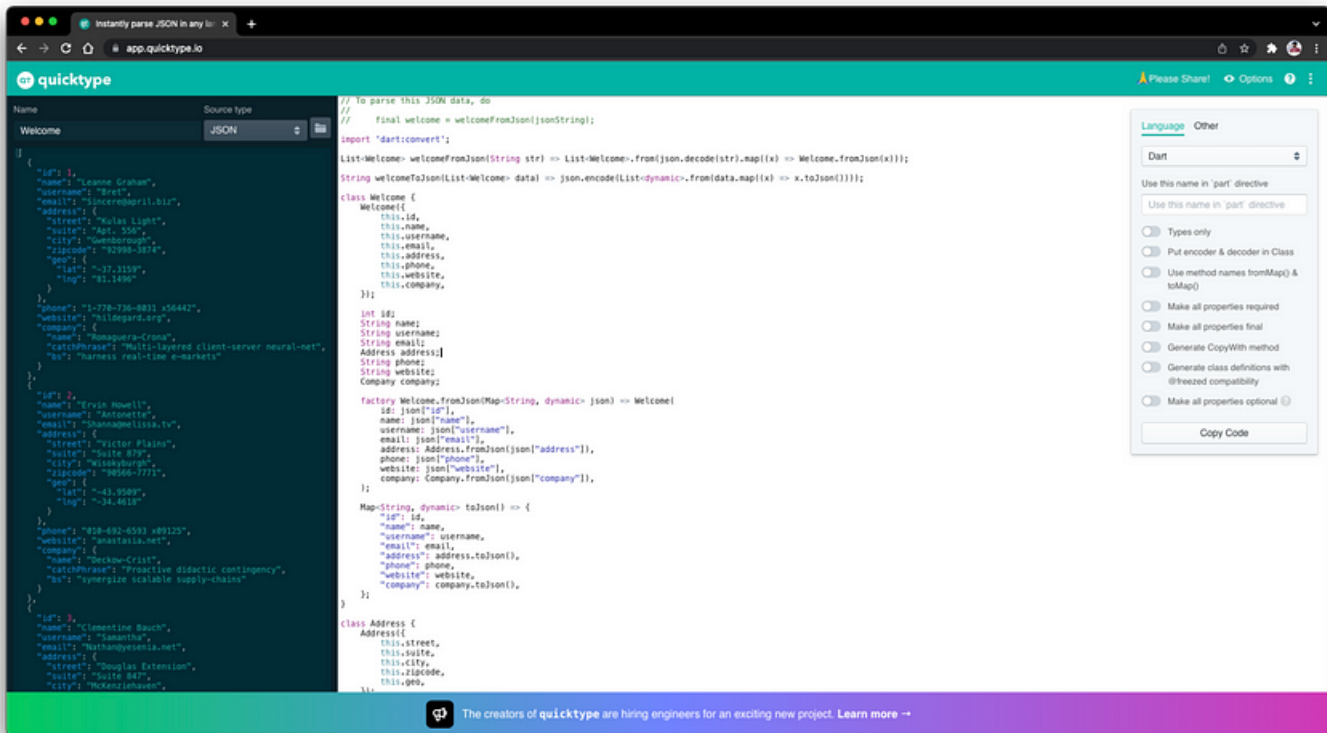
Access data directly:

```
final data = json[0]['id'];
```

In this, we tried to access the `id` of 0th record of the JSON response. This is easy only when you have limited records and entries.

Now, to create a model class, first, you will need the entire JSON response that you are getting from the API. To get that, just go to <https://jsonplaceholder.typicode.com/users> and you will be able to obtain the entire JSON response as this is a public API. Now, in the case of private API or custom backend APIs, either it will be available in the documentation or you can use [Postman](#) to hit the API and get the response.

Now, once you have the response, you can create the model class using a simple method. Go to <https://app.quicktype.io> and on the left side paste the JSON response. In the options on the right side, select `Dart` and your model class will be generated! You can then change the name of the class.



Your model class file will look something like the following:

```

1  import 'dart:convert';
2
3  List<UserModel> userModelFromJson(String str) =>
4    List<UserModel>.from(json.decode(str).map((x) => UserModel.fromJson(x)));
5
6  String userModelToJson(List<UserModel> data) =>
7    json.encode(List<dynamic>.from(data.map((x) => x.toJson())));
8
9  class UserModel {
10    UserModel({
11      required this.id,
12      required this.name,
13      required this.username,
14      required this.email,
15      required this.address,
16      required this.phone,
17      required this.website,
18      required this.company,
19    });
20
21    int id;

```

```

22     String name;
23     String username;
24     String email;
25     Address address;
26     String phone;
27     String website;
28     Company company;
29
30     factory UserModel.fromJson(Map<String, dynamic> json) => UserModel(
31         id: json["id"],
32         name: json["name"],
33         username: json["username"],
34         email: json["email"],
35         address: Address.fromJson(json["address"]),
36         phone: json["phone"],
37         website: json["website"],
38         company: Company.fromJson(json["company"]),
39     );
40
41     Map<String, dynamic> toJson() => {
42         "id": id,
43         "name": name,
44         "username": username,
45         "email": email,
46         "address": address.toJson(),
47         "phone": phone,
48         "website": website,
49         "company": company.toJson(),
51     }
52
53     class Address {
54         Address({
55             required this.street,
56             required this.suite,
57             required this.city,
58             required this.zipcode,
59             required this.geo,
60         });
61
62         String street;
63         String suite;
64         String city;
65         String zipcode;
66         Geo geo;
67
68         factory Address.fromJson(Map<String, dynamic> json) => Address(
69             street: json["street"],
70             suite: json["suite"],
71             city: json["city"],
72             zipcode: json["zipcode"],
73             geo: Geo.fromJson(json["geo"]),
74         );
75
76         Map<String, dynamic> toJson() => {
77             "street": street,
78             "suite": suite,
79             "city": city,

```



```
80         "zipcode": zipcode,
81         "geo": geo.toJson(),
82     };
83 }
84
85 class Geo {
86     Geo({
87         required this.lat,
88         required this.lng,
89     });
90
91     String lat;
92     String lng;
93
94     factory Geo.fromJson(Map<String, dynamic> json) => Geo(
95         lat: json["lat"],
96         lng: json["lng"],
97     );
98
99     Map<String, dynamic> toJson() => {
100         "lat": lat,
101         "lng": lng,
102     };
103 }
104
105 class Company {
106     Company({
107         required this.name,
108         required this.catchPhrase,
109
110         required this.bs,
111     });
112
113     String name;
114     String catchPhrase;
115     String bs;
116
117     factory Company.fromJson(Map<String, dynamic> json) => Company(
118         name: json["name"],
119         catchPhrase: json["catchPhrase"],
120         bs: json["bs"],
121     );
122
123     Map<String, dynamic> toJson() => {
124         "name": name,
125         "catchPhrase": catchPhrase,
126         "bs": bs,
127     };
128 }
```

Step 5: Create a file that handles the API call and write specific methods to fetch data and parse it.

Now, create a file called `api_service.dart` that will handle the API calls.

```
1  import 'dart:developer';
2
3  import 'package:http/http.dart' as http;
4  import 'package:rest_api_example/constants.dart';
5  import 'package:rest_api_example/model/user_model.dart';
6
7  class ApiService {
8    Future<List<UserModel>>> getUsers() async {
9      try {
10        var url = Uri.parse(ApiConstants.baseUrl + ApiConstants.usersEndpoint);
11        var response = await http.get(url);
12        if (response.statusCode == 200) {
13          List<UserModel> _model = userModelFromJson(response.body);
14          return _model;
15        }
16      } catch (e) {
17        log(e.toString());
18      }
19    }
20  }
```

8

Now, in the above file, we have created a function called `getUsers` that returns `List<UserModel>`. The first step is to hit the GET HTTP request. The next step is to check, whether the API call was successful or not. That can be checked using `response.statusCode`. If the API call will be successful, `statusCode` will be 200. If the `statusCode` is 200, we then convert the JSON (`response.body`) into a `List<UserModel>` using the method `userModelFromJson` which is available in the model class that we created and then return the model object.

Step 6: Use the data in your app.

We have created all the required files and methods for the API, which we can call our app backend. Now it's time to load this data on our UI. We won't be doing much, just a method call and load that result in UI.

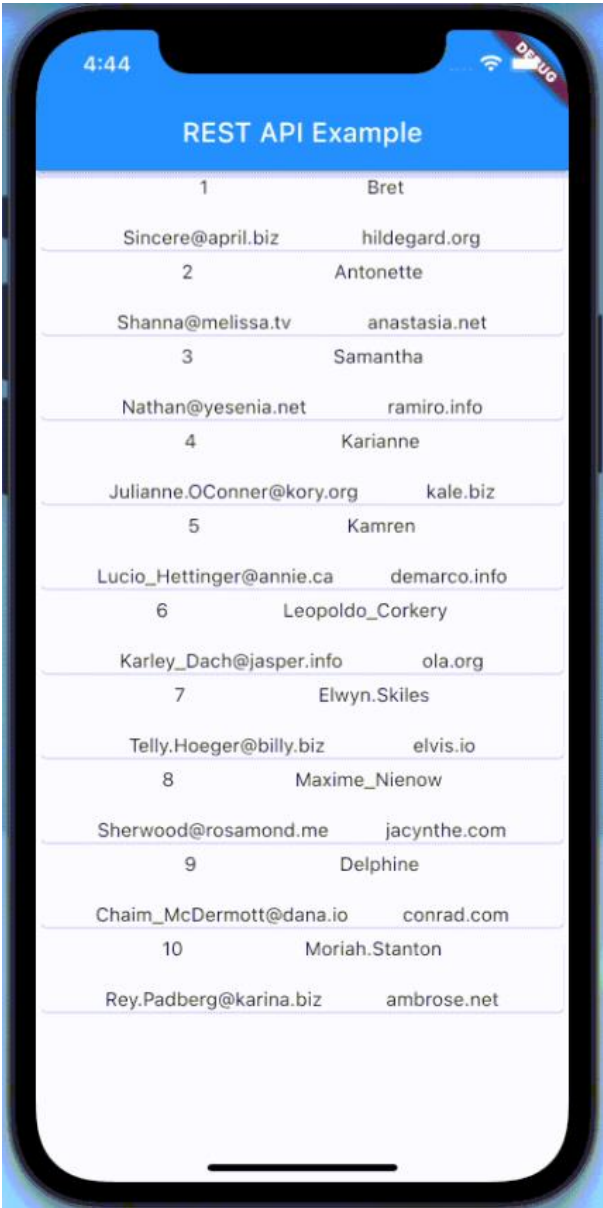
```

1  import 'package:flutter/material.dart';
2  import 'package:rest_api_example/model/user_model.dart';
3  import 'package:rest_api_example/services/api_service.dart';
4
5  class Home extends StatefulWidget {
6    const Home({Key? key}) : super(key: key);
7
8    @override
9    _HomeState createState() => _HomeState();
10 }
11
12 class _HomeState extends State<Home> {
13   late List<UserModel> _userModel = [];
14   @override
15   void initState() {
16     super.initState();
17     _getData();
18   }
19
20   void _getData() async {
21     _userModel = (await ApiService().getUsers())!;
22     Future.delayed(const Duration(seconds: 1)).then((value) => setState(() {}));
23   }
24
25   @override
26   return Scaffold(
27     appBar: AppBar(
28       title: const Text('REST API Example'),
29     ),
30     body: _userModel == null || _userModel!.isEmpty
31       ? const Center(
32         child: CircularProgressIndicator(),
33       )
34       : ListView.builder(
35         itemCount: _userModel!.length,
36         itemBuilder: (context, index) {
37           return Card(
38             child: Column(
39               children: [
40                 Row(
41                   mainAxisAlignment: MainAxisAlignment.spaceEvenly,
42                   children: [
43                     Text(_userModel![index].id.toString()),
44                     Text(_userModel![index].username),
45                   ],
46                 ),
47                 const SizedBox(
48                   height: 20.0,
49                 ),
50                 Row(
51                   mainAxisAlignment: MainAxisAlignment.spaceEvenly,
52                   children: [
53                     ],
54                   ),
55                 ],
56             ),
57           ),
58         ],
59       ),
60     );
61   },
62 );
63 );
64 }
65 }

```

Now, in the above code, we created an object of `List<UserModel>` and then in the `initState` we called a method to get the data from our API. In the method, I have added `Future.delayed`, just so that we can simulate the API time response. This API that we are using is a fake one and hence it's pretty fast. However, real APIs aren't that fast. So it becomes our duty to show a loader until the API response is available. Once the response is available, we rebuild the UI to show the data. Here, I have just shown a few of the data from the model class.

Output



REST API Example		
1	Bret	
	Sincere@april.biz	hildegard.org
2	Antonette	
	Shanna@melissa.tv	anastasia.net
3	Samantha	
	Nathan@yesenia.net	ramiro.info
4	Karianne	
	Julianne.OConner@kory.org	kale.biz
5	Kamren	
	Lucio_Hettinger@annie.ca	demarco.info
6	Leopoldo_Corkery	
	Karley_Dach@jasper.info	ola.org
7	Elwyn.Skiles	
	Telly.Hoeger@billy.biz	elvis.io
8	Maxime_Nienow	
	Sherwood@rosamond.me	jacynthe.com
9	Delphine	
	Chaim_McDermott@dana.io	conrad.com
10	Moriah.Stanton	
	Rey.Padberg@karina.biz	ambrose.net

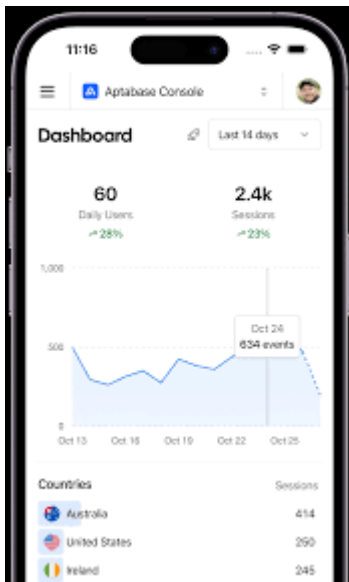
Common APIs that every APP needs nowadays

- API calls for customer authentication.

Subscribe

☒ Remember me for faster sign in

- Analytics



- Backend
- Push Notification APIs (Auth0 etc)
- Integrating Payment Gateways (google pay, paypal) (optional)
- Maps & Location APIs (optional)

Solution:

API Response

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
    "company": {
      "name": "Romaguera-Crona",
      "catchPhrase": "Multi-layered client-server neural-net",
      "bs": "harness real-time e-markets"
    }
  },
  {
    "id": 2,
    "name": "Ervin Howell",
    "username": "Antonette",
    "email": "Shanna@melissa.tv",
    "address": {
      "street": "Victor Plains",
      "suite": "Suite 879",
      "city": "Wisokyburgh",
      "zipcode": "90566-7771",
      "geo": {
        "lat": "-43.9509",
        "lng": "-34.4618"
      }
    },
    "phone": "010-692-6593 x09125",
    "website": "anastasia.net",
    "company": {
      "name": "Deckow-Crist",
      "catchPhrase": "Proactive didactic contingency",
      "bs": "synergize scalable supply-chains"
    }
  },
  {
    "id": 3,
    "name": "Clementine Bauch",
    "username": "Samantha",
    "email": "Nathan@yesenia.net",
    "address": {
      "street": "Douglas Extension",
      "suite": "Suite 847",
      "city": "McKenziehaven",
      "zipcode": "59590-4157",
      "geo": {
        "lat": "-68.6102",
        "lng": "-47.0653"
      }
    },
    "phone": "1-463-123-4447",
    "website": "ramiro.info",
    "company": {
      "name": "Romaguera-Jacobson",
      "catchPhrase": "Face to face bifurcated interface",
      "bs": "e-enable strategic applications"
    }
  },
  {
    "id": 4,
    "name": "Patricia Lebsack",
    "username": "Karianne",
    "email": "Julianne.OConner@kory.org",
    "address": {
      "street": "Hoeger Mall",
      "suite": "Apt. 692",
      "city": "South Elvis",
      "zipcode": "53919-4257",
      "geo": {
```

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^2.30.0
  firebase_auth: ^4.17.0
  cloud_firestore: ^4.15.0
  shared_preferences: ^2.2.2
  particles_flutter: ^0.1.4
  simple_animations: ^5.0.0
  image_picker: ^1.1.2
  firebase_storage: ^11.7.7
  firebase_messaging: ^14.9.4
  http: ^0.13.5
```

api_service.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
  Future<List<dynamic>> fetchUsers() async {
    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/users'),
    );

    if (response.statusCode == 200) {
      return jsonDecode(response.body);
    } else {
      throw Exception("API Failed");
    }
  }
}
```

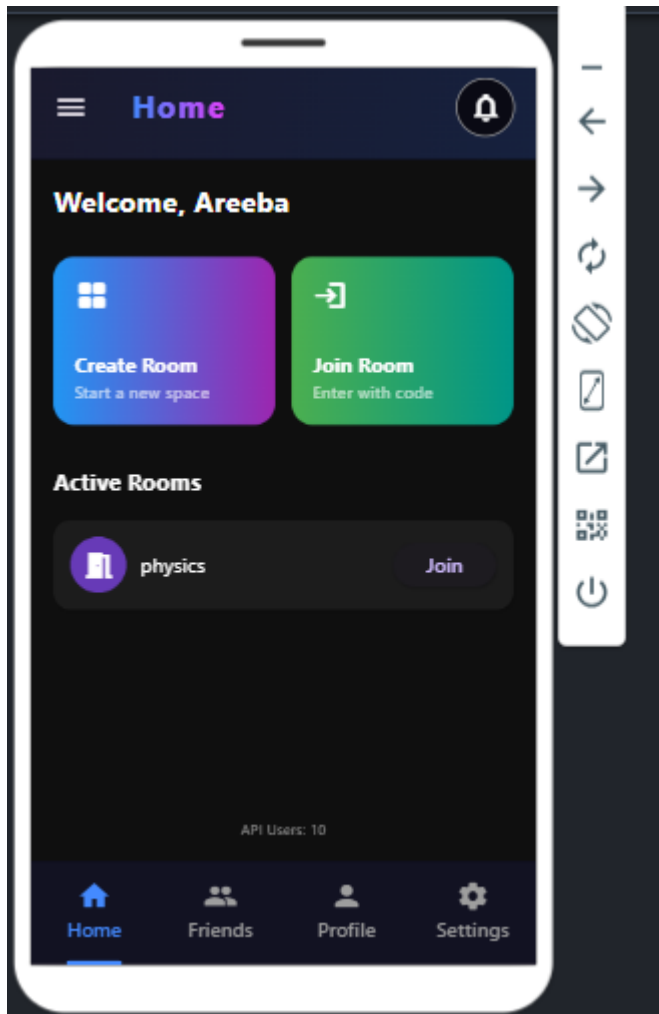
HomeScreen

```
List apiUsers = [];  
bool isLoadingApi = false;  
  
@override  
void initState() {  
  super.initState();  
  fetchUserName();  
  loadApiData(); // API call added  
}
```

```
// ===== API FUNCTION =====  
void loadApiData() async {  
  setState(() {  
    isLoadingApi = true;  
  });  
  
  try {  
    apiUsers = await ApiService().fetchUsers();  
    print("API Users Loaded: ${apiUsers.length}");  
  } catch (e) {  
    print("API Error: $e");  
  }  
  
  setState(() {  
    isLoadingApi = false;  
  });  
}
```

```
Align(  
  alignment: Alignment.bottomCenter,  
  child: Padding(  
    padding: const EdgeInsets.only(top: 8),  
    child: Text(  
      isLoadingApi  
        ? "Loading API..."  
        : "API Users: ${apiUsers.length}",  
      style: const TextStyle(  
        color: Colors.grey,  
        fontSize: 10,  
      ), // TextStyle  
    ), // Text  
  ), // Padding  
, // Align
```


UI Output



Console Output

```
≡ API Users Loaded: 10
```